

Тема 11. Работа с дати и часове. Модул за дата и час. Операции с дати

Работа с дати и часове

Модул за дата и час

Основната функционалност за работа с дати и часове е концентрирана в модула `datetime` под формата на следните класове:

- Дата;
- Време;
- Време за среща.

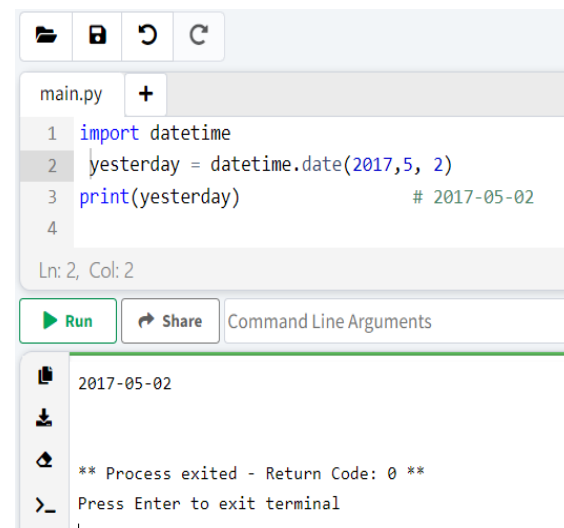
Клас дата

За да се работи с дати, ще се използва класа дата, който е дефиниран в модула `datetime`. За да създадем обект за дата, може да се използва конструктора за дата, който приема три параметъра последователно: година, месец и ден.
`date(year, month, day)`

Пример: Да се създаде конкретна дата:

```
import datetime
```

```
yesterday = datetime.date(2017,5, 2)  
print(yesterday)           # 2017-05-02
```

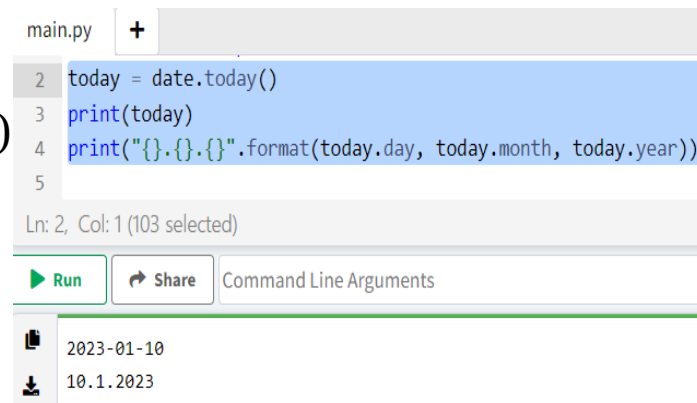


```
main.py +  
1 import datetime  
2 yesterday = datetime.date(2017,5, 2)  
3 print(yesterday)           # 2017-05-02  
4  
Ln: 2, Col: 2  
  
Run Share Command Line Arguments  
  
2017-05-02  
  
** Process exited - Return Code: 0 **  
>_ Press Enter to exit terminal  
|
```

Тема 11. Работа с дати и часове. Модул за дата и час. Операции с дати

За получаване на текущата дата, може да се използва метода `today()`:

```
today = date.today()  
print(today)  
print("{}.{}.{}".format(today.day, today.month, today.year))
```



The screenshot shows a code editor with a file named `main.py`. The code contains three lines: `today = date.today()`, `print(today)`, and `print("{}.{}.{}".format(today.day, today.month, today.year))`. The third line is highlighted in blue. Below the code editor, there are buttons for `Run` and `Share`, and a text field for `Command Line Arguments`. At the bottom, the output of the program is displayed, showing the date `2023-01-10` and the formatted date `10.1.2023`.

Може да се прилагат свойствата за ден, месец, година.

Клас време

Класът по време е отговорен за работата с времето. Създаде на времеви обект става чрез използване на конструкторът му:

`time([hour] [, min] [, sec] [, microsec])`

Конструкторът приема часове, минути, секунди и микросекунди последователно. Всички параметри са незадължителни и ако не се предаде нито един параметър, тогава съответната стойност ще бъде инициализирана на нула.

Тема 11. Работа с дати и часове. Модул за дата и час. Операции с дати

```
from datetime import time
```

```
current_time = time()  
print(current_time)    # 00:00:00
```

```
current_time = time(16, 25)  
print(current_time)    # 16:25:00
```

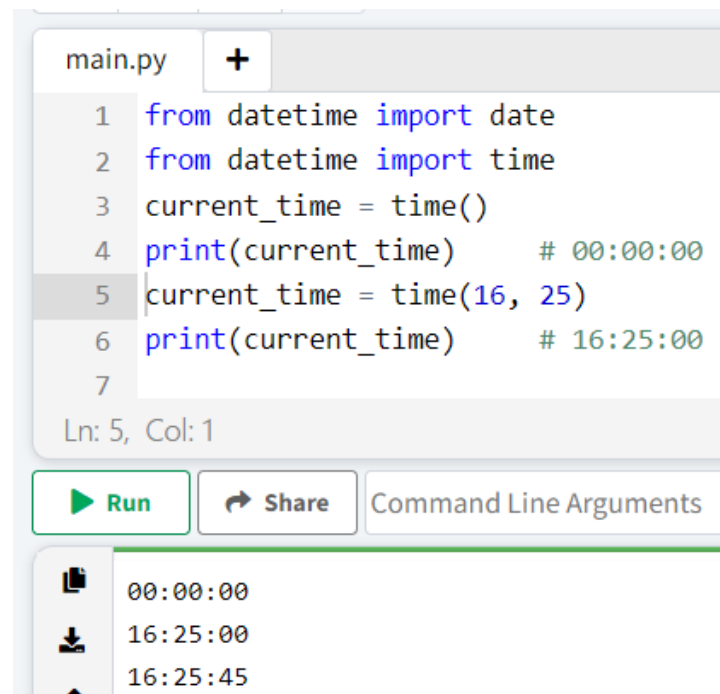
```
current_time = time(16, 25, 45)  
print(current_time)    # 16:25:45
```

Клас дата и час

Класът `datetime` от едноименния модул съчетава възможностите за работа с дата и час. За да създаде обект `datetime`, може да се използва следния конструктор:

`datetime(year, month, day [, hour] [, min] [, sec] [, microsec])`

Задължителни са първите три параметъра, представляващи година, месец и ден. Останалите са по избор и ако не посочим стойност за тях, те се инициализират на нула по подразбиране.



```
main.py +  
1 from datetime import date  
2 from datetime import time  
3 current_time = time()  
4 print(current_time)    # 00:00:00  
5 current_time = time(16, 25)  
6 print(current_time)    # 16:25:00  
7  
Ln: 5, Col: 1  
Run Share Command Line Arguments  
00:00:00  
16:25:00  
16:25:45
```

Тема 11. Работа с дати и часове. Модул за дата и час. Операции с дати

```
from datetime import datetime
deadline = datetime(2017, 5, 10)
# 2017-05-10 00:00:00
print(deadline)
```

```
deadline = datetime(2017, 5, 10, 4, 30)
# 2017-05-10 04:30:00
print(deadline)
```

```
1 from datetime import datetime
2 deadline = datetime(2017, 5, 10)
3 print(deadline)      # 2017-05-10 00:00:00
4 deadline = datetime(2017, 5, 10, 4, 30)
5 print(deadline)      # 2017-05-10 04:30:00
```

Ln: 4, Col: 1

 Run  Share Command Line Arguments

```
2017-05-10 00:00:00
2017-05-10 04:30:00
```

За да се получи текущата дата и час, може да се извика метода `now()`:

```
from datetime import datetime
now = datetime.now()
print(now)
print("{}.{}.{} {}:{}".format(now.day,
now.month, now.year, now.hour, now.minute))
print(now.date())
print(now.time())
```

```
main.py +
1 from datetime import datetime
2 now = datetime.now()
3 print(now)
4 print("{}.{}.{} {}:{}".format(now.day,
5 now.month, now.year, now.hour, now.minute))
6 print(now.date())
7 print(now.time())
8 |
9
```

Ln: 8, Col: 1

 Run  Share Command Line Arguments

```
2023-01-10 09:01:03.490567
10.1.2023 9:1
2023-01-10
09:01:03.490567
```

Като се използват свойствата `ден`, `месец`, `година`, `час`, `минута`, `секунда`, може да се получат индивидуални стойности за дата и час. И чрез методите `date()` и `time()` може да се получат датата и часа поотделно.

Преобразуване от низ към дата

От функционалността на класа `datetime` трябва да се отбележи методът `strptime()`, който ще позволи да се анализира низ и да се преобразува в дата. Този метод приема два параметъра:

`strptime(str, format)`

Първият параметър към `str` е низова дефиниция на датата и часа, а вторият параметър е формат, който определя как различните части на датата и часа са подредени в този низ. За да се определи формата, може да се използват следните кодове:

- **%d**: ден от месеца като число
- **%m**: редовен месец
- **%y**: година, като 2 числа
- **%Y**: година, като 4 числа
- **%H**: час в 24-часов формат
- **%M**: минута
- **%S**: второ

Различните формати изглеждат по следния начин:

```
from datetime import datetime
deadline = datetime.strptime("22/05/2017",
"%d/%m/%Y")
print(deadline)    # 2017-05-22 00:00:00
```

```
deadline = datetime.strptime("22/05/2017 12:30",
"%d/%m/%Y %H:%M")
print(deadline)    # 2017-05-22 12:30:00
```

```
deadline = datetime.strptime("05-22-2017 12:30",
"%m-%d-%Y %H:%M")
print(deadline)    # 2017-05-22 12:30:00
```

```
1 from datetime import datetime
2 deadline = datetime.strptime("22/05/2017",
3 "%d/%m/%Y")
4 print(deadline)    # 2017-05-22 00:00:00
5 deadline = datetime.strptime("22/05/2017 12:30",
6 "%d/%m/%Y %H:%M")
7 print(deadline)    # 2017-05-22 12:30:00
8 deadline = datetime.strptime("05-22-2017 12:30",
9 "%m-%d-%Y %H:%M")
10 print(deadline)    # 2017-05-22 12:30:00
11
```

n: 8, Col: 1

Run

Share

Command Line Arguments

```
2017-05-22 00:00:00
2017-05-22 12:30:00
2017-05-22 12:30:00
```

Операции с дати

Форматиране на дати и часове

И двата класа предоставят метода `strptime(format)` за форматиране на обекти за дата и час. Този метод приема само един параметър, който определя формата за преобразуване на датата или часа. За да се определи формата, може да се използва един от следните форматни кодове:

Тема 11. Работа с дати и часове. Модул за дата и час. Операции с дати

- **%a**: съкращение за деня от седмицата. Например Wed - от думата Wednesday (английските имена се използват по подразбиране)
- **%A**: пълен ден от седмицата, например - сряда
- **%b**: съкращение за името на месеца
- **%B**: пълно име на месеца, например - октомври
- **%d**: ден от месеца, подплатен с нула, например - 01
- **%m**: номер на месеца, подплатен с нула, например - 05
- **%y**: година, като 2 числа
- **%Y**: година, като 4 числа
- **%H**: час в 24-часов формат, например - 13
- **%I**: час в 12-часов формат, например 01
- **%M**: минута
- **%S**: секунда
- **%f**: микросекунда
- **%p**: показалец - AM/PM
- **%c**: дата и час, форматирувани за текущата локация
- **%x**: дата, форматирувана за текущата локация
- **%X**: време, форматирувано за текущата локация

Тема 11. Работа с дати и часове. Модул за дата и час. Операции с дати

Използване на различни формати:

```
from datetime import datetime
now = datetime.now()
print(now.strftime("%Y-%m-%d"))
print(now.strftime("%d/%m/%Y"))
print(now.strftime("%d/%m/%y"))
print(now.strftime("%d %B %Y (%A)"))
print(now.strftime("%d/%m/%y %I:%M"))
```

```
1 from datetime import datetime
2 now = datetime.now()
3 print(now.strftime("%Y-%m-%d"))
4 print(now.strftime("%d/%m/%Y"))
5 print(now.strftime("%d/%m/%y"))
6 print(now.strftime("%d %B %Y (%A)"))
7 print(now.strftime("%d/%m/%y %I:%M"))
```

Ln: 3, Col: 45

 Run  Share Command Line Arguments

```
2023-01-10
10/01/2023
10/01/23
10 January 2023 (Tuesday)
10/01/23 09:06
```

Когато се показват имената на месеците и дните от седмицата, английските имена се използват по подразбиране. Ако трябва да се използва текущата локализация, то тогава може да се зададе предварително с помощта на локалния модул:

```
from datetime import datetime
import locale
locale.setlocale(locale.LC_ALL, "")
now = datetime.now()
print(now.strftime("%d %B %Y (%A)"))
```

```
1 from datetime import datetime
2 import locale
3 locale.setlocale(locale.LC_ALL, "")
4 now = datetime.now()
5 print(now.strftime("%d %B %Y (%A)"))
```

Ln: 5, Col: 45

 Run  Share Command Line Arguments

```
10 January 2023 (Tuesday)
```


Тема 11. Работа с дати и часове. Модул за дата и час. Операции с дати

Добавяне и изваждане на дати и часове

Често, когато се работи с дати, се налага добавяне на определен период от време към вече съществуваща дата или, обратно, да се извади определен период. За извършване на такива операции в класът **timedelta** е дефиниран модула **datetime**. Всъщност този клас дефинира определен период от време. Може да се използва конструктора `timedelta`, за да се дефинира времеви интервал:

`timedelta([days] [, seconds] [, microseconds] [, milliseconds] [, minutes] [, hours] [, weeks])`

Предаването на дни, секунди, микросекунди, милисекунди, минути, часове и седмици на конструктора.

Пример: Да се дефинират няколко периода:

```
from datetime import timedelta
three_hours = timedelta(hours=3)
print(three_hours)
three_hours_thirty_minutes = timedelta(hours=3,
minutes=30)
two_days = timedelta(2)
print(two_days)
two_days_three_hours_thirty_minutes = timedelta(days=2,
hours=3, minutes=30)
print(two_days_three_hours_thirty_minutes)
```

```
1 from datetime import timedelta
2 three_hours = timedelta(hours=3)
3 print(three_hours)
4 three_hours_thirty_minutes = timedelta(hours=3,
5 minutes=30)
6 two_days = timedelta(2)
7 print(two_days)
8 two_days_three_hours_thirty_minutes = timedelta(days=2,
9 hours=3, minutes=30)
10 print(two_days_three_hours_thirty_minutes)
```

n: 10, Col: 43

Run

Share

Command Line Arguments

```
3:00:00
2 days, 0:00:00
2 days, 3:30:00
```

Тема 11. Работа с дати и часове. Модул за дата и час. Операции с дати

Като се използва `timedelta`, може да се добавят или изваждат дати.

Пример: Да се добави към конкретна дата два дни:

```
from datetime import timedelta, datetime
now = datetime.now()
print(now)
two_days = timedelta(2)
in_two_days = now + two_days
print(in_two_days)
```

```
1 from datetime import timedelta, datetime
2 now = datetime.now()
3 print(now)
4 two_days = timedelta(2)
5 in_two_days = now + two_days
6 print(in_two_days)
```

n: 6, Col: 36

Run

Share

Command Line Arguments

```
2023-01-10 09:14:16.113585
2023-01-12 09:14:16.113585
```

Пример: Да се определи, колко е бил часът преди 10 часа и 15 минути, спрямо текущото време:

```
from datetime import timedelta, datetime
now = datetime.now()
till_ten_hours_fifteen_minutes = now - timedelta(hours=10,
minutes=15)
print(till_ten_hours_fifteen_minutes)
```

```
1 from datetime import timedelta, datetime
2 now = datetime.now()
3 till_ten_hours_fifteen_minutes = now - timedelta(hours=10,
4 minutes=15)
5 print(till_ten_hours_fifteen_minutes)
```

n: 1, Col: 1 (171 selected)

Run

Share

Command Line Arguments

```
2023-01-09 23:00:53.121256
```

timedelta свойства

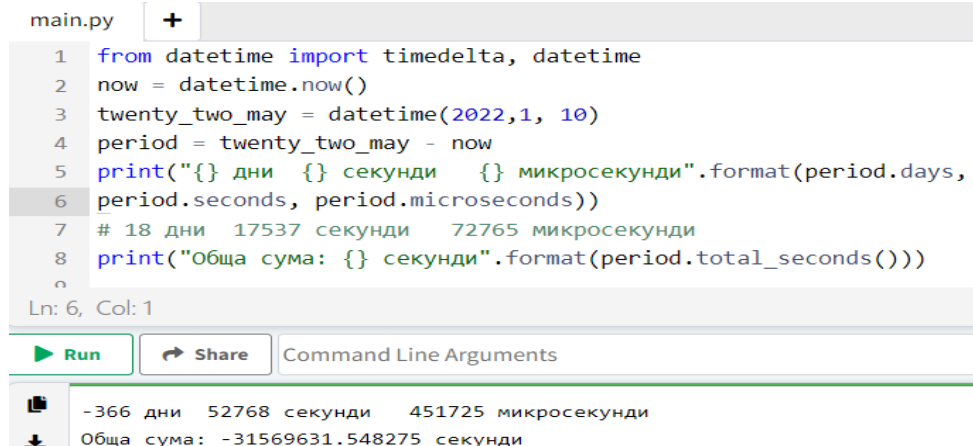
Класът timedelta има няколко свойства, които може да се използват, за да се получи времевият интервал:

- **дни:** връща броя на дните;
- **секунди:** връща броя на секундите;
- **микросекунди:** връща броя на микросекунди.

В допълнение, методът total_seconds() връща общия брой секунди, който включва дни, секунди и микросекунди.

Пример: Да се определи период от време между две дати:

```
from datetime import timedelta, datetime
now = datetime.now()
twenty_two_may = datetime(2017, 5, 22)
period = twenty_two_may - now
print("{} дни {} секунди {} микросекунди".format(period.days,
period.seconds, period.microseconds))
print("Обща сума: {} секунди".format(period.total_seconds()))
```



The screenshot shows a Python IDE with a file named 'main.py'. The code in the file is as follows:

```
1 from datetime import timedelta, datetime
2 now = datetime.now()
3 twenty_two_may = datetime(2022,1, 10)
4 period = twenty_two_may - now
5 print("{} дни {} секунди {} микросекунди".format(period.days,
6 period.seconds, period.microseconds))
7 # 18 дни 17537 секунди 72765 микросекунди
8 print("Обща сума: {} секунди".format(period.total_seconds()))
9
```

The IDE shows the cursor at line 6, column 1. Below the code editor, there are buttons for 'Run' and 'Share', and a field for 'Command Line Arguments'. The output of the script is displayed at the bottom:

```
-366 дни 52768 секунди 451725 микросекунди
Обща сума: -31569631.548275 секунди
```

Тема 11. Работа с дати и часове. Модул за дата и час. Операции с дати

Сравнение на дати

Точно като низовете и числата, датите могат да се сравняват с помощта на стандартни оператори за сравнение.

Пример:

```
from datetime import datetime
now = datetime.now()
deadline = datetime(2017, 5, 22)
if now > deadline:
```

```
    print("Срокът за изпълнение на проекта изтече")
elif now.day == deadline.day and now.month == deadline.month and now.year == deadline.year:
    print("Краен срок на проекта днес")
else:
    period = deadline - now
    print("Наляво {} дни".format(period.days))
```

```
1 from datetime import datetime
2 now = datetime.now()
3 deadline = datetime(2017, 5, 22)
4 if now > deadline:
5     print("Срокът за изпълнение на проекта изтече")
6 elif now.day == deadline.day and now.month == deadline.month and now.year == deadline.year:
7     print("Краен срок на проекта днес")
8 else:
9     period = deadline - now
10    print("Наляво {} дни".format(period.days))
11
```

Ln: 11, Col: 1



Run



Share

Command Line Arguments



Срокът за изпълнение на проекта изтече

Използвана литература:

- [1]. Joakim Sundnes, Introduction to Scientific Programming with Python, Simula Springer Briefs on Computing, 2020
- [2]. Brian Heinold, A Practical Introduction to Python Programming, Department of Mathematics and Computer Science Mount St. Mary's University, 2012
- [3]. David Mertz, Functional Programming in Python, O'Reilly Media, 2015
- [4]. Steven Lott, Functional Python Programming, Published by Packt Publishing Ltd., 2015
- [5]. Mark Lutz, Learning Python, Fourth Edition, O'Reilly Media, 2009